

How a Merchant Payout Works

A walkthrough of the code path that moves a tuck shop's daily sales out to their mobile money wallet — from the admin endpoint down to the signed request Yo Uganda receives.

Prepared for Markmiro, Miro Technologies **Scope** assessment only, no code changed

Dated 2 Aug 2026

TEST MODE

No YO_USERNAME, or APP_ENV isn't "production" — every Yo call is faked. No money, real or sandbox, moves.

SANDBOX, SIGNED

APP_ENV=production against Yo's sandbox host. Real RSA signatures, real HTTP calls, fake money.

LIVE

Same code path, production Yo + production DATABASE_URL. Real UGX leaves the float account.

01 — TRIGGER

Two endpoints, one core

Both routes in `app/routes/reports.py` funnel into the same worker, `_process_merchant_payout()` — there is no separate code path for "manual" vs "automatic."

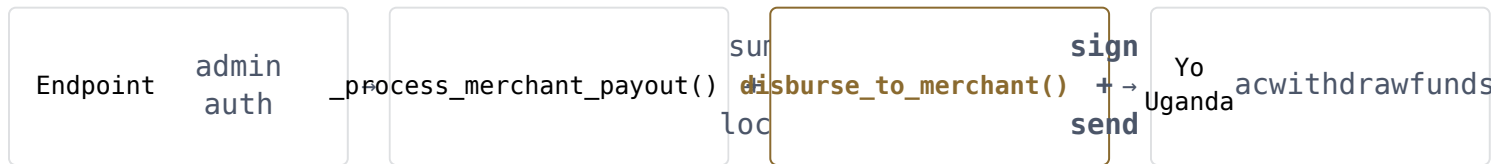
POST `/reports/school/{school_id}/payout`

One school, on demand. Requires an admin JWT (`get_current_admin`) and `assert_school_access` — a school-scoped admin can only pay out their own school; a super admin (`school_id IS NULL`) can pay out any school.

POST `/reports/settlements/auto`

Every school at once. Requires admin auth **and** a `secret` query param matching `SETTLEMENT_SECRET` — which falls back to a hardcoded default if that env var is unset on Railway. Not the endpoint to use for a single test payout.

Request flow



02 — IDEMPOTENCY

The Payout row is a reservation, not a receipt

One row per `(merchant_id, payout_date)`, enforced by a unique constraint — this is what makes it safe to call the endpoint twice by accident. It is written **before** Yo is ever contacted, not after.

- 1 Sum the merchant's `completed payment`-type transactions for the target date. Zero sales → stop, nothing to pay.
- 2 Row-lock (`with_for_update`) any existing `Payout` for this merchant + date — this is what keeps two concurrent triggers from racing each other.
- 3 Existing row already **pending** or **sent** → stop, report "skipped." Only a **failed** row is reused for a retry.
- 4 No row yet → insert one as **pending** and commit. **The reservation exists before Yo is called** — a crash mid-call still leaves a row to reconcile against.
- 5 Call `disburse_to_merchant()`.
- 6 Update the same row to **sent** (with Yo's reference) or **failed**, based on the response.

03 — THE WIRE

What `disburse_to_merchant` sends and expects

`app/momo.py` uses Yo's `acwithdrawfunds` method. In test mode it fakes success and touches the network not at all. For a real call:

XML REQUEST BODY

```
<AutoCreate>
  <Request>
    <APIUsername>...</APIUsername>
    <APIPassword>...</APIPassword>
    <Method>acwithdrawfunds</Method>
    <Amount>{merchant_total}</Amount>
    <Account>{merchant.momo_phone}</Account>
    <Narrative>Daily payout to {merchant_name}</Narrative>
    <ExternalReference>{uuid4}</ExternalReference>
    <PublicKeyAuthenticationNonce>{uuid4.hex}</PublicKeyAuthenticationNonce>
    <PublicKeyAuthenticationSignatureBase64>{signature}</PublicKeyAuthenticationSignatureBase64>
  </Request>
</AutoCreate>
```

signature = base64(RSA-SHA1_sign(SHA1_hex(APIUsername + Amount + Account + Narrative + ExternalReference + Nonce))) – order is fixed by Yo §4.1, never reorder.

Yo answers with XML — `parse_yo_response()` flattens it to a dict. `reports.py` treats `Status == "OK"` as success; `TransactionStatus: INDETERMINATE` means neither — resolve it with `verify_transaction()`, never assume.

What decides real vs. faked

YO_USERNAME	APP_ENV	RESULT
unset	any	Faked — <code>_is_test_mode()</code> is true
set	<code>≠ "production"</code>	Faked — still test mode
set	<code>"production"</code>	Real signed call, to whatever <code>YO_API_URL</code> resolves to

Note what's absent from that table: nowhere does it check which database is in use. `APP_ENV` and `DATABASE_URL` are two unrelated switches — see the risk flags below.

04 — PROCEDURE

Running one real sandbox payout, safely

Entirely local. Never run against the deployed Railway service for this test.

- 1 **Stand up a throwaway Postgres** — `createdb school_wallet_sandbox_test`. This never touches the real, deployed database.
- 2 **Point `DATABASE_URL` at it, locally only** — export it in your shell for this run, don't edit the committed `.env`.
- 3 **Set `APP_ENV=production`** plus your sandbox `YO_USERNAME` / `YO_PASSWORD` / `YO_PRIVATE_KEY`, for this local process only.
- 4 **Seed one School, one Merchant, one Transaction** — merchant's `momo_phone` should be whatever test MSISDN Yo's sandbox docs specify; the transaction should be `completed`, type `payment`, dated today, small (e.g. 100 UGX).
- 5 **Call the scoped endpoint** — `POST /reports/school/{id}/payout` with an admin token from that same local database.
- 6 **Read the console line** — `disburse_to_merchant` prints `Yo Uganda payout: {Status} ({TransactionStatus}) - {merchant} - ref {uuid}`. Confirm `YO_API_URL` actually resolved to the sandbox host before you fire.

05 — VERIFY

What confirms it actually worked

- ✓ One row in

```

payouts
(merchant_id,
today)
status
=
'sent'
yo_reported
completed_at

```

- ✓ Yo's own response —

```

Status
TransactionStatus
StatusMessage

```

Message captured from the call.

- ✓ `TransactionStatus: INDETERMINATE`

→ `verify_transaction(ref)`
call

before
concluding
anything.

✓ Re-running the same trigger for the same merchant/date now returns

outcomes, our idempotency check, for free.
"skipped"

✓ No new row in

transaction only reads sales, it never writes one.

06 — FLAGS

Where this can go wrong

DANGER APP_ENV and DATABASE_URL are independent switches

Flipping `APP_ENV=production` only changes whether Yo calls are real — it has no relationship to which Postgres instance the app writes to. Do this in any shell that still carries the real Railway `DATABASE_URL`, and a "just testing sandbox" run writes a live `Payout` row and calls Yo for real against real merchant data.

CAUTION YO_API_URL is overridable

Defaults to the sandbox host, but it's read from the environment. A shell that inherited a Railway-exported production value would sign and send a real withdraw request without anything in the code stopping it. Print and confirm the resolved URL before firing.

CAUTION settlements/auto has a public fallback secret

`SETTLEMENT_SECRET` falls back to a hardcoded value if unset on Railway, and that value is already in git history. Use the scoped `/reports/school/{id}/payout` for a single test — never the all-schools endpoint.

CAUTION No pre-debit / reversal step

Yo's own \$4.1 guidance is to debit the merchant's balance on our side first, then call, then reverse on failure.

`disburse_to_merchant` doesn't do this — it has no internal ledger safety net beyond the `Payout` row's status field.